

Q1. LSTM: main gates and their tasks

ISHAAN JAIN

An LSTM (Long Short term memory) cell is designed to let recurrent networks keep information for long time spans while avoiding vanishing/exploding gradients. The LSTM core uses a cell state C_t and a hidden state h_t . The behaviour is controlled by three main gates plus a candidate cell update:

Gate and Roles

1. Forget gate f_t

- Purpose - decide which parts of previous cell state C_{t-1} to keep or forget.

- Equation: $f_t = \sigma(W_f \cdot [h_{t-1}, u_t] + b_f)$

- Effect: elementwise multiply with C_{t-1} so values near 0 are forgotten, near 1 are retained.

2. Input Gate i_t (update gate)

- Purpose - decide which components of the newly computed candidate C_t to add into the cell gate

- Equation - $i_t = \sigma(W_i \cdot [h_{t-1}, u_t] + b_i)$

3. Candidate cell state C_t

- Purpose - propose new content to add to the cell state (usually passed through $\tanh$)

- Equation - $C_t = \tanh(W_c \cdot [h_{t-1}, u_t] + b_c)$

4. Cell state update

- Combine forget + input

$$C_t = f_t \odot C_{t-1} + i_t \odot C_t$$

5. Output gate o_t and hidden state h_t

- Purpose - decide which parts of the cell state to expose as the hidden output.

- Equations - $o_t = \sigma(W_o \cdot [h_{t-1}, u_t] + b_o)$

$$h_t = o_t \odot \tanh(C_t)$$

- Intuition - cell keeps internal memory C_t ; o_t gates how much of that memory passes out as h_t .

Importance

- The forget gate prevents irrelevant old information from accumulating and lets LSTM discard state memory.

- The input gate + candidate let the cell add new information
- The output gate controls what internal memory should influence the next layer / time-step or final output.
- The cell state c_t with near additive updates avoids repeated multiplication by small gradients, so long-range gradients can flow - this is why LSTMs handle long dependencies better than vanilla RNNs.

Q2. Transformer Architecture

Transformer replaced recurrence with attention mechanism that operate on all positions in the sequence simultaneously.

A transformer block contains -

- Multi-head self-attention layer (allowing the model to attend to different positions and representation subspaces).
- Position-wise feedforward network (two linear layers with activation).
- Residual connections and layer normalization around sublayers.

Used in NLP because -

1. Parallelism - self-attention computes dependencies for all token pairs in a layer simultaneously - training can be parallelized across positions.
2. long-range dependencies - Attention links any token pair, easily modeling distant relationships.
3. Scalability - Scales effectively with depth, width and large data, enhancing transferability.
4. Pretraining + Fine-tuning - Large scale pretraining with later fine-tuning boosts task performance.
5. Expressive representations - Multihead attention captures diverse syntactic, semantic and positional patterns.

Self-Attention (Scaled Dot-Product Attention)

Each input token embedding X is projected into:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Attention Formula -

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- QK^T : Similarity scores between tokens
- Division by $\sqrt{d_k}$: stabilizes gradients
- Softmax: converts scores to attention weights
- Multiplying by V : gives weighted context vectors

Multi-head Attention -

Multiple attention heads different token relationships (semantic, syntactic, positional) then concatenate outputs for richer representations.

Positional Encoding -

Since attention is order-agnostic, positional encodings (sinusoidal or learned) are added to embeddings to encode word order.

Advantages -

- Captures pairwise relations directly.
- Enables parallel training
- Multi-heads gives diverse contextual clues
- Flexible across NLP tasks (classification, QA, translation, generation)

Q3.

(i) Output Feature Map Size

Input = $32 \times 32 \times 3$

Filters = 8, kernel = 5×5 , Stride = 1, Padding = 0

$$\text{outsize} = \left\lfloor \frac{N-f}{s} \right\rfloor + 1$$

Apply to height and width -

$$\text{outheight} = (32-5)/1 + 1 = 28$$

$$\text{outwidth} = (32-5)/1 + 1 = 28$$

Depth = number of filters = 8

Output feature map - $28 \times 28 \times 28$

(2) Trainable Parameters

Each filter: $5 \times 5 \times 3 = 75$ weights + 1 bias
 $= 76$ parameters

Total for eight filters $= 76 \times 8$
 $= 608$ parameters

Why ReLU over sigmoid/Tanh

Aspect	Relu	sigmoid/tanh
Gradient flow	No saturation for positive inputs $\rightarrow$ avoid vanishing gradients	Gradients vanish for large
Activation sparsity	outputs 0 for negative $\rightarrow$ sparse, efficient	Dense activations
Computation	$\text{ReLU}(x) = \max(0, x)$; very cheap	expensive, exponentials
Convergence	faster, better optima in deep CNN	slower convergence
Nature	Piecewise linear, easy to optimize	Non linear solution

Qn. Autoencoder vs PCA

Aspect	PCA	AutoEncoder (AE)
Type	Linear dimensionality reduction	Neural network-based, can be non-linear
Goal	Find orthogonal directions (principal components) Capturing Max variance	Encode-decode input to minimize reconstruction loss
Computation	Closed form via eigen decomposition / SVD	Trained iteratively via gradient descent
Representation	Linear global subspace	Non-linear manifold
Extensions	None beyond linear	Denosing, sparse, variational, over/undercomplete AEs

Domain
Relation
labels in
Target
Example

Same /
different

Yes

BERT →
sentiment

Different
domains

No / few

MNIST →
SVHN

Same /
different

No

SIMCLR pretrain
→ clustering